

动态规划

动态规划适合解决的问题模型：

1. **最优子结构**：指的是，问题的最优解包含子问题的最优解。反过来说就是，**我们可以通过子问题的最优解，推导出问题的最优解**。对应到动态规划问题模型上可以理解为，**后面阶段的状态可以通过前面状态推导出来**。
2. **无后效性**：无后效性，有两层含义，第一层含义是，**在推导后面阶段状态的时候，我们只关心前面阶段的状态值，不关心这个状态是怎么一步步推导出来的**。第二层含义是，某阶段状态一旦确定，就不受之后阶段的决策影响。即**已经求解的子问题，不会再受到后续决策的影响**。
3. **重复子问题**：不同的决策序列，到达某个相同的阶段时，可能会产生重复的状态。即存在子问题进行了重复计算。我们可以用空间将这些子问题的解存储下来，避免重复求解相同的子问题，从而提升效率。

基本思路：

动态规划本质上是利用历史记录，来避免我们的重复计算。而这些历史记录（动态规划表），我们需要一些变量来保存，一般是用一维数组或者二维数组来保存。下面我们来看看动态规划的基本思路。

1. 确定状态：将原问题划分为若干阶段，每个阶段对应若干个子问题，提取这些子问题的特征（称之为状态）；并且一般是从最后一步从底层一步一步往上逆推的。
2. 确定状态转移方程：寻找每一个状态的可能决策，或者说是各状态间的相互转移方式（用数学的语言描述就是 状态转移方程）。
3. 确定开始以及边界条件。
4. 按顺序求解每一个阶段的问题。

例子：

先用一个很简单的样例来**解释**一下吧

给定一个长度为 n 的数组 a ，现在需要从这 n 个数里找一个最大的数。

这个简单的问题用动态规划去解决的话，大概是这样的：

假如我们知道前 $i-1$ 项的最大值，那么**前 i 项的最大值**就为**前 $i-1$ 项的最大值**与**第 i 项的值**最大的那个，即：由前 $i-1$ 项的最优解可以得到前 i 项的最优解

用 $dp[i]$ 表示**前 i 项的最大值**，那么状态转移方程为： $dp[i] = \max(dp[i-1], a[i])$

那么求解问题就可以这么写：

```
for(int i=1;i<=n;i++)//dp[0]=0
    dp[i] = max(dp[i-1],a[i]);
cout<<dp[n];
```

现在，问题难度再难一点

定义一个区间的区间值为该区间所有项之和

给定一个长度为 n 的数组 a ($a[i]$ 可以为负)，现在要求最大的区间值

例如 $[1, -5, 3, 8, -1, 2, -5]$ 的最大区间和为 $3 + 8 - 1 + 2 = 12$

这个问题用动态规划来解决的话，大概思路是：

假如我们已知以第 $i-1$ 项为结尾的区间的最大值，那么求解以第 i 项为结尾的区间的最大值时，第 i 项要么与前面那个区间合并为一个区间，要么自己独立为一个区间

设 $dp[i]$ 表示以第 i 项为结尾的区间的最大值，那么状态转移方程为 $dp[i] = \max(dp[i-1] + a[i], a[i])$ ，该问题的答案就为所有 $dp[i]$ 的最大值

那么这个问题可以这么解决：

```
for(int i=1; i<=n; i++) // dp[0]=0
    dp[i] = max(dp[i-1]+a[i], a[i]);

int ans=-10000000;
for(int i=1; i<=n; i++)
    ans=max(ans, dp[i]);
cout<<ans;
```

那现在，我们来解决01背包问题 <https://www.acwing.com/problem/content/2/>

有一个容量为 V 的背包，还有 n 个物体，每个物体的体积为 $v[i]$ ，价值为 $w[i]$ 。现在忽略物体实际几何形状，我们认为只要背包的剩余容量大于等于物体体积，那就可以装进背包里。

求背包能放入物体的总价值最大为多少

大致思路：

假设 $dp[i][j]$ 为前 i 个物体放入体积为 j 的背包的最优解

对于 $dp[i][j]$ ：

要么 $dp[i][j] = dp[i-1][j]$ ，即前 i 个物体放入体积为 j 的背包的最优解 = 前 $i-1$ 个物体放入体积为 j 的背包的最优解；

要么 $dp[i][j] = dp[i-1][j-v[i]] + w[i]$ ，即前 i 个物体放入体积为 j 的背包的最优解 = 前 $i-1$ 个物体放入体积为 (j - 第 i 个物品的体积) 的背包的最优解 + 第 i 个物品的价值。

那么

那么状态转移方程为：

当 $v[i] \leq j$ 时， $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-v[i]] + w[i])$

当 $v[i] > j$ 时， $dp[i][j] = dp[i-1][j]$

求解问题时：

```

for(int i=1;i<=n;i++)
{
    for(int j=1;j<=v;j++)
    {
        if(v[i]<=j)//如果第i个物品能放下
            dp[i][j]=max(dp[i-1][j],dp[i-1][j-v[i]]+w[i]);
        else
            dp[i][j]=dp[i-1][j];
    }
}
cout<<dp[n][v];

```

使用滚动数组优化空间复杂度：

dp[j]表示大小为j的背包的最优解

```

for(int i=1;i<=n;i++)
{
    for(int j=v;j>=v[i];j--)//这个for循环注意一下，一定是从大到小循环，从小到大就
    {
        //变为了无限背包问题，因为物体会被重复取
        dp[j]=max(dp[j],dp[j-v[i]]+w[i]);
    }
}
cout<<dp[v];

```